Dramatis

“the persons of the drama”

A manual for reading a story by the company it keeps — installing it, using it, living with it, and removing it.

Written for authors, editors, translators and literary scholars. No programming knowledge is assumed. Everything you have to type is given in full.

Dramatis 0.1.0 (pre-alpha) · A Kestora Labs product · Apache-2.0 ·

Describes the application as built through Phase 4. Features marked *“not yet”* are planned and named so you know they are coming, not missing by accident.

Contents

PART I

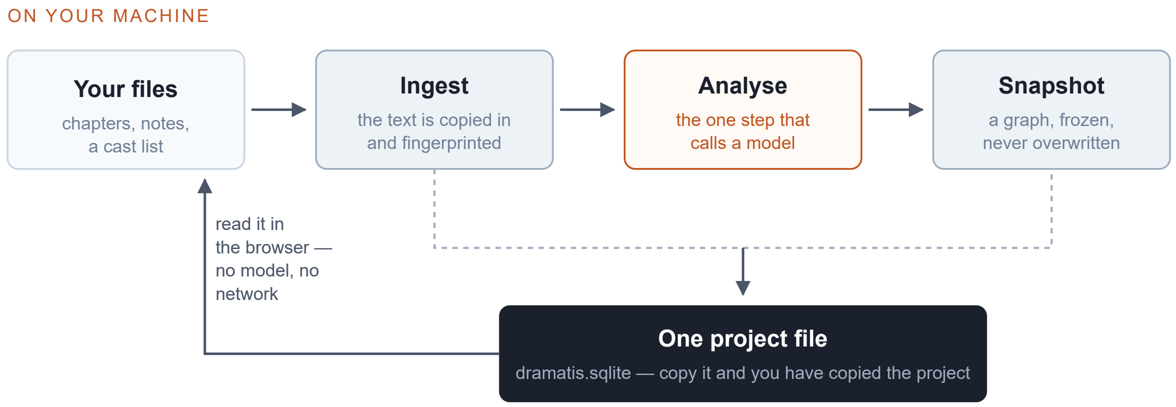
Before you start

Four short chapters. What the thing is, what it promises, the handful of words it uses in a particular way, and the one idea that everything else rests on. Fifteen minutes here will save you an afternoon later.

1 What Dramatis is, and is not

Dramatis reads a narrative work and draws its characters as a network: who is in it, who is close to whom, and what in the text says so. It saves each reading as a dated, unchangeable snapshot, so that when you revise the work you can put the old picture beside the new one and see what moved.

That is the whole of it. It is a reading instrument, not a writing tool. It never edits your manuscript, never suggests a plot, and never offers an opinion about quality. It answers questions of shape: *who carries this book? which relationships am I actually writing, as against the ones I meant to write? when I cut chapter nine, who fell out of the story with it?*



The shape of the tool. Your files go in; a fingerprinted copy is stored; a model reads that copy once and the result is frozen as a snapshot. Everything after that — opening it, reading it, comparing it, exporting it — happens on your machine with no model and no network. Only the orange step ever contacts anything.

What it is good at

- **Seeing a cast whole.** A novel has more people in it than anyone can hold in mind. A picture of them, sized by how much of the book they carry, is a different object from a list of names.

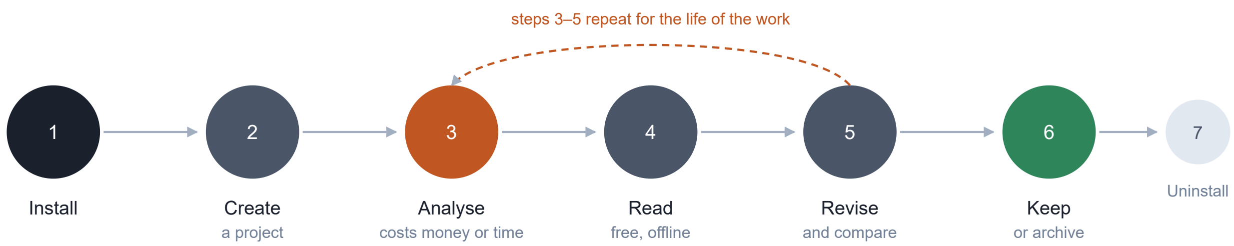
- **Watching a revision land.** Rewrite a chapter, run the analysis again, and Dramatis will tell you which relationships strengthened, which weakened, and which disappeared — and, crucially, whether the change belongs to your rewrite or to the analysis having been done differently.
- **Catching the gap between the plan and the page.** If you keep a cast list or a series bible, Dramatis reads it separately and shows you the relationships you have described but never dramatised, and the ones that took over the book without ever being in the plan.
- **Citable claims.** Every edge in the graph is backed by quotations checked letter-by-letter against your text. You can click any of them and land on the sentence.

What it deliberately is not

- **Not a writing tool.** It reads and reflects. It has no editor and never writes to your manuscript files.
- **Not tied to one author's filing method.** It will not assume that a folder called `draft-2` is a second draft, or that a file called `cast.md` is a cast list. It proposes; you confirm.
- **Not a general knowledge graph.** Characters and their relationships only. Places, objects and events are out of scope.
- **Not a model.** Dramatis ships no artificial intelligence of its own. It orchestrates a model that you supply — either a paid service you hold an account with, or a free one running on your own computer.
- **Not a service.** There are no accounts, no cloud storage, no sharing features and no subscription. It runs on your machine and nowhere else.

The life of a project

This manual follows that life from end to end. Steps 3 to 5 are the loop you will actually live in: write, re-read, compare.



Seven stages, and where they are covered. Installing is Part II; creating, analysing and reading are Part III; the loop of revising and comparing is chapters 19 and 20 and is worked through end to end in Part IV; keeping and removing are chapters 25 and 30.

> A WORD ABOUT THE VERSION
>
> Dramatis is **pre-alpha**. It works, it is thoroughly tested, and the parts described in this manual do what they say. But it is early software: the installation is a developer-style one for now, there is no double-clickable installer yet, and features named in chapter 29 are genuinely absent rather than hidden. Treat a Dramatis project as valuable but not yet as a system of record.

2 The three promises

Three commitments are built into the design rather than into the marketing. They are worth knowing because they explain a lot of behaviour that would otherwise look pedantic.

Your text stays where you put it

There is no telemetry, no analytics and no phone-home of any kind. The only outbound network connection Dramatis will ever make is to the model provider you name, at the moment you ask for an analysis. If you choose a model running on your own machine (chapter 7), nothing leaves the machine at all — you can unplug the network cable and the analysis still completes.

Everything else — opening a project, drawing the graph, following evidence into the source, comparing two drafts, exporting — works with no credential and no network, permanently and by design. This is not a fallback mode; it is the ordinary way the application runs.

Nothing is claimed without a quotation

Every relationship in the graph carries quotations from your text, and every one of them has been checked character-by-character against the source before it was allowed into the record. If the model invents a line, the claim resting on it is *rejected* — not flagged, not shown with a warning, but removed. Where a quotation is genuine but attached to the wrong passage, it is moved to the right one rather than thrown away.

The single concession is whitespace: a quotation spanning a line break in a hard-wrapped file still counts as verbatim. Nothing else is forgiven — not a changed comma, not a straightened quotation mark, not a difference of case.

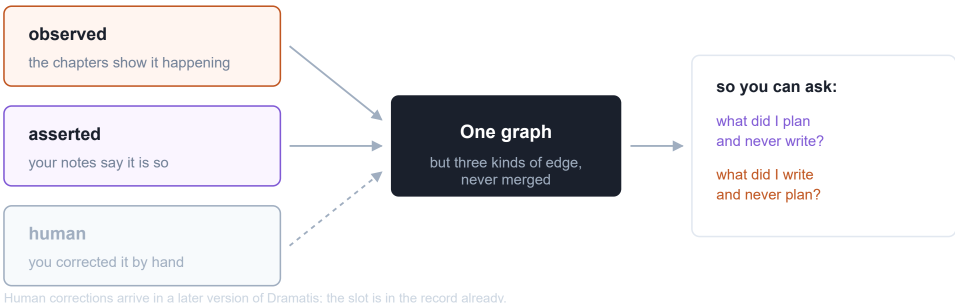
Nothing is ever overwritten

A snapshot is immutable. Analysing the same text again does not replace the earlier reading; it adds a second one beside it. Ingesting an edited chapter does not replace the old chapter; it records a new revision and keeps the old one, because a snapshot taken last month cites text that must still be there to be cited. Your project file grows; it does not churn.

3 Twelve words you will need

Dramatis uses a small number of ordinary words in a precise way. Learn these and the rest of the manual reads easily. A fuller glossary is at the back.

Project	One file, holding everything: your texts, the cast, and every analysis you have ever run. Copy the file and you have copied the project entire. It is normally called `dramatis.sqlite` and lives in the folder with your manuscript.
Collection	The circle inside which characters are shared. A project holds one collection. If you put two novels of a series into the same project, a character appearing in both is *one* character, which is what a series wants. An unrelated book belongs in its own project.
Work	A novel, a play, a season — one thing that gets revised over time.
Document	One file. A chapter, a cast list, an appendix. Each has a *role*: it either enacts the story or describes it.
Text revision	The exact state of your manuscript at one moment, fingerprinted. Every time you bring the folder in again, Dramatis records a new revision and tells you which files changed.
Analysis run	One reading: which model, which instructions, which settings. Two runs with the same configuration are the same reading and can be compared.
Snapshot	A graph, permanently bound to *one* text revision and *one* analysis run. This pairing is the heart of the design; chapter 4 explains why.
Character	A node. Carries a canonical name and every surface form the text used for them — "Elizabeth", "Miss Eliza Bennet", "Lizzy".
Relation	An edge between two characters, with a weight, a type or two, and its evidence.
Weight and basis	How strong an edge is, *and* what the number counts. A weight without its basis is meaningless, so the two always travel together and Dramatis refuses to compare weights measured on different bases.
Provenance	Where a claim came from: **observed** (the narrative enacts it), **asserted** (your reference material states it), or **human** (you entered or corrected it). Never mixed.
Structure map	Dramatis's proposal about what your folder holds — which file is story, which is notes, which is a later draft of which — with the reason for each guess, waiting for you to confirm or correct it.



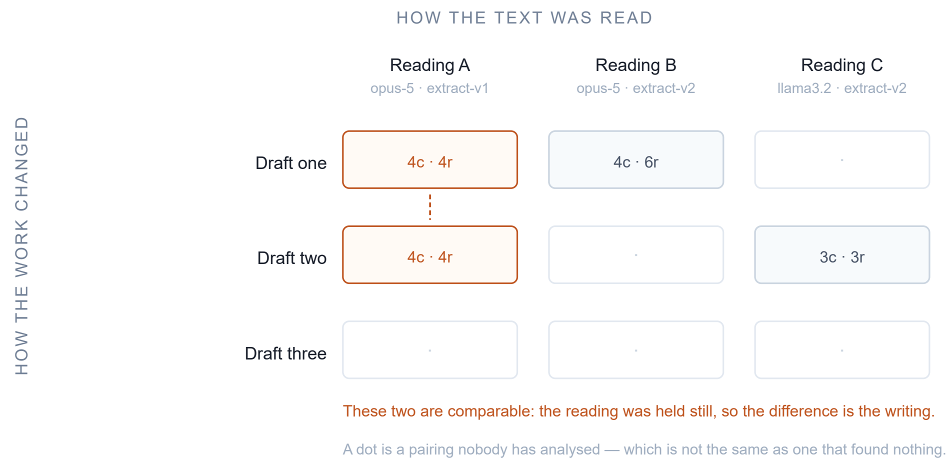
Three sources, one graph, never merged. An edge that your notes declare and an edge that your chapters enact are two different claims about the same pair. Kept apart, they let you ask the two questions on the right. Merged into one number they would answer neither.

4 The two time axes

This is the one idea in Dramatis worth reading twice. Everything about the way snapshots are stored and compared follows from it.

A character graph can change for two entirely different reasons. Either **the work changed** — you rewrote a chapter, cut a subplot, gave a minor character three more scenes. Or **the reading changed** — you ran the analysis with a better model, or at a higher effort setting, or with revised instructions.

If a tool cannot tell you which of those happened, its comparisons are worthless. You would have no way of knowing whether your rewrite worked or whether you simply paid for a smarter reader. So Dramatis never collapses the two. A snapshot is bound to a text revision *and* an analysis run, both always recorded, and the browser lays your snapshots out on a grid with one axis for each.



Reading the grid. Down a column, the reading is held still and the text varies: any difference is your writing. Across a row, the text is held still and the reading varies: any difference is the analysis. A snapshot in a different row *and* a different column can be blamed on neither, and Dramatis will say so in plain words rather than pretend.

In practice this means one habit is worth forming: when you re-analyse after a rewrite, tell Dramatis to hold the reading exactly as it was last time. There is a single option for it, `--like`, and chapter 19 shows it in use.

PART II

Installing

Dramatis is pre-alpha, so installation is still a developer-style one: a few commands in a terminal window rather than a double-clickable installer. A signed desktop application is planned. What follows assumes you have never used a terminal and explains every step.

5 What you need first

A terminal

A terminal is a window where you type commands instead of clicking. You already have one.

- **macOS** — press `⌘ Space`, type *Terminal*, press Return.
- **Windows** — press the Start key, type *PowerShell*, press Return.
- **Linux** — you know where it is.

Throughout this manual, anything shown in a dark box is typed into that window and followed by Return. Lines beginning with `#` are comments for you, not commands — you can leave them out.

Python 3.11 or later

Dramatis is written in Python. Check whether you already have a suitable version:

```
python3 --version
```

If that prints `Python 3.11.x` or higher, you are set. If it prints something older, or an error, install a current Python from python.org/downloads — take the default options, and on Windows tick *"Add Python to PATH"* on the first screen of the installer. On Windows the command is usually `python` rather than `python3`.

Node.js 20 or later

The browser view is built from source at installation time, which needs Node.js. Check with:

```
node --version
```

If it is missing, install the LTS release from nodejs.org. You will never need to think about Node again after installation.

Git

Used once, to fetch the source. `git --version` will tell you whether you have it; if not, git-scm.com/downloads.

Somewhere for the model to come from

Analysis needs a language model. You have two choices and they have quite different consequences; chapter 7 lays them out. You do not need to decide before installing.

6 Installing, step by step

> **DO NOT INSTALL FROM PYPI**
> There is an unrelated package also called `dramatis` in the public Python index. `pip install dramatis` will fetch that stranger's package, not this one. Install from the source checkout exactly as below.

1 · Fetch the source

Choose where it should live and clone it there:

```
# macOS and Linux
cd ~
git clone https://github.com/kestora-labs/dramatis.git
cd dramatis
```

```
# Windows PowerShell
cd $HOME
git clone https://github.com/kestora-labs/dramatis.git
cd dramatis
```

2 · Make a private workspace for it

A *virtual environment* is a folder holding this application's dependencies so they cannot disturb anything else on your computer. Create and activate one:

```
# macOS and Linux
python3 -m venv .venv
source .venv/bin/activate
```

```
# Windows PowerShell
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Your prompt will now begin with `(.venv)`. That is how you know the workspace is active. It stays active until you close the window; whenever you come back to Dramatis in a new terminal, run the `activate` line again first.

> IF WINDOWS REFUSES TO RUN THE ACTIVATE SCRIPT
> PowerShell blocks scripts by default. Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer *Yes*, and try again.

3 · Install the application

The square brackets ask for two optional extras: `anthropic`, the adapter for the hosted model, and `serve`, the browser view. Include both; they are small and you will want them.

```
pip install -e ".[anthropic,serve]"
```

4 · Build the browser view

Two commands, once, taking a minute or so:

```
npm ci --prefix web
npm --prefix web run build
```

This turns the source of the browser interface into files the Dramatis server can hand to your browser. If you skip it, every command still works but `dramatis serve` will show a page telling you the client has not been built.

5 · Confirm

```
dramatis --version
```

Should print something like `dramatis 0.1.0.dev0`. If it prints *command not found*, the virtual environment is not active — go back to step 2.

7 Choosing where the thinking happens

Analysis is the one step that needs a language model, and you decide which. The choice is not permanent — it is made per analysis — but it has real consequences for cost, quality and privacy, so make it deliberately.

	A HOSTED MODEL (ANTHROPIC)	A MODEL ON YOUR MACHINE (OLLAMA)
Your text	Is sent to the provider, in passages, as the analysis runs.	Never leaves the computer. Works with the network cable out.
Cost	Paid per use, on your own account with the provider.	Free. Costs electricity and time.
Quality	Currently much better, especially at recognising that two names are one person.	Usable; depends entirely on the model you download.
Speed	Minutes for a novel.	Minutes to hours, depending on your machine.
Set-up	An account and an API key.	Install one application, download one model.

Option A — a hosted model

Create an account at console.anthropic.com and generate an API key. Then make it available to Dramatis by putting it in the environment before you run an analysis:

```
# macOS and Linux
export ANTHROPIC_API_KEY=sk-ant-...
```

```
# Windows PowerShell
$env:ANTHROPIC_API_KEY = "sk-ant-..."
```

That setting lasts until you close the window. To make it permanent, add the same line to your shell profile — on macOS that is usually `~/.zshrc`.

> AN API KEY IS A PASSWORD
>
> Anyone holding it can spend your money. Do not paste it into a document, an email, or a file you will later commit to version control.

Option B — a model on your own machine

Install Ollama — it is an ordinary application with a normal installer for macOS, Windows and Linux. Then download a model:

```
ollama pull llama3.1
```

That is the whole set-up. There is no key, no account, and no charge. From then on, add `--provider ollama` to any analysis and the manuscript stays on your machine.

> A LOCAL MODEL IS ONLY LOCAL IF IT IS LOCAL
>
> Ollama can be pointed at another computer. If yours is, Dramatis notices and prints a warning saying your text is about to leave the machine, so you find out before the analysis rather than after it.

Which to pick

If you are analysing a published or public-domain text, use the hosted model: it is better and the privacy question does not arise. If you are analysing an unpublished manuscript that you are not comfortable sending anywhere, use Ollama and accept a rougher reading. Many people do both — a local pass while drafting, a hosted pass when it matters.

8 Checking that it worked

Dramatis ships with a hand-authored example graph. Validating it exercises the schema and its checks without needing a model, a key, or a network:

```
dramatis validate fixtures/a/snapshot.json
```

```
ok    fixtures/a/snapshot.json
```

And the same command on a deliberately broken example:

```
dramatis validate fixtures/a/invalid/self-relation.json
```

```
FAIL  fixtures/a/invalid/self-relation.json — 1 problem
      [reference] /relations/0: a relation may not join a character to itself
```

If both behave that way, your installation is sound.

9 Installing with Docker instead

If you already use Docker, there is an image that bundles the application, the browser view and everything it needs. It is an alternative to Part II above, not an addition.

```
docker build -t dramatis .
docker run --rm -p 127.0.0.1:7373:7373 -v "$PWD/project:/data" dramatis
```

The project file lands at `./project/dramatis.sqlite` on your machine, and the interface is at `http://127.0.0.1:7373`.

> WHY THE ADDRESS IS WRITTEN OUT IN FULL
>
> Inside a container, “this machine only” is unreachable from outside it, so the server inside binds to everything. Publishing the port to `127.0.0.1:7373` rather than `7373` is what keeps your manuscript off the office network. Write the address out; the short form is not the same.

PART III

Using it

Everything the application currently does, in the order you will meet it: making a project, telling Dramatis what your files are, running a reading, and then the several ways of looking at the result. Screenshots are from real analyses of *Pride and Prejudice*.

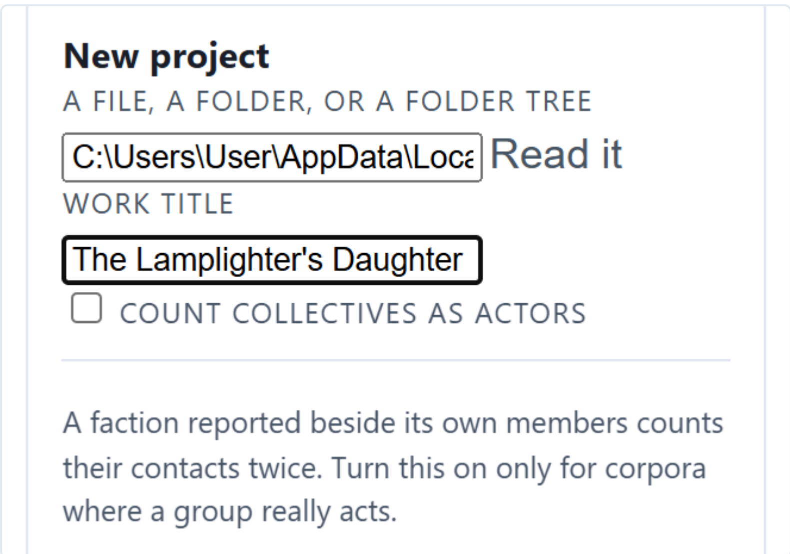
10 Making a project in the browser

The friendlier of the two routes. You point the server at a project file that does not exist yet, open it in your browser, and build the project by clicking.

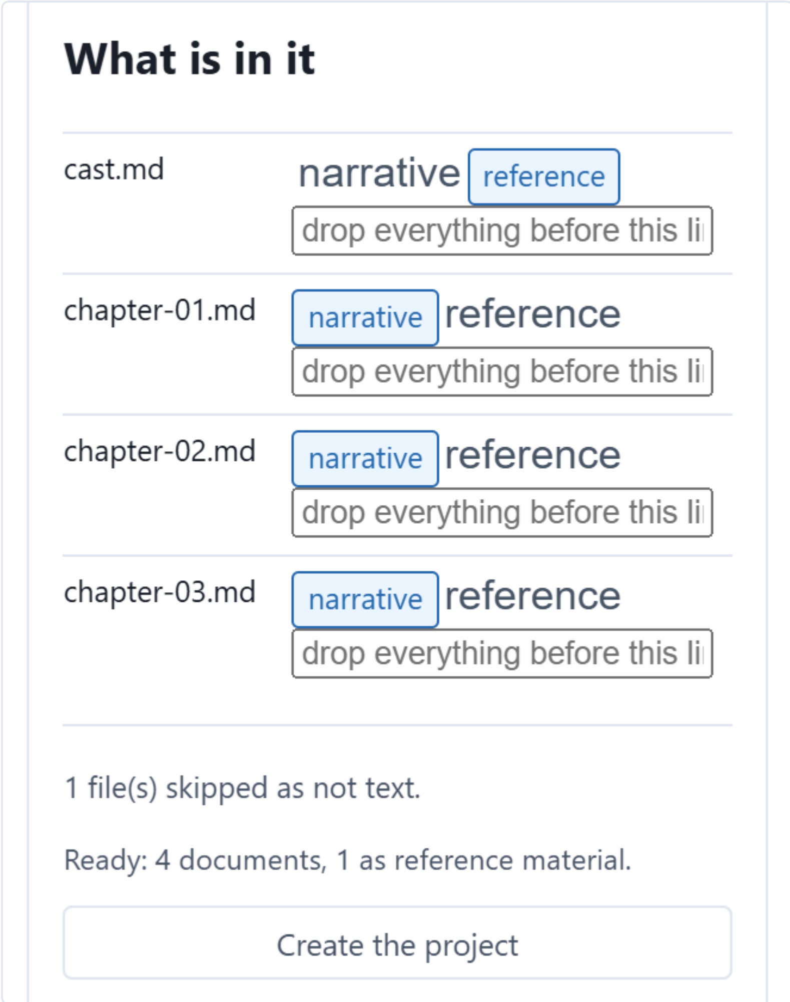
Put your manuscript files in a folder, then, from inside that folder:

```
cd ~/writing/my-novel
dramatis serve --store dramatis.sqlite
```

Nothing is created yet. Dramatis says as much, prints an address, and waits. Open `http://127.0.0.1:7373` in your browser. Because the project holds nothing, the **New project** panel opens itself.



Where the work is, and what to call it. A path, a title, and one question about the whole study. Pressing *Read it* reads the folder; it calls no model and writes nothing.



The screen that matters. Dramatis lists what it found and asks one question about each file: does this *enact* the story, or *describe* it? Nothing is assumed from the filename — you are asked, and asked once. It will not let you create the project until every file has an answer.

The panel has four things in it:

- **The source.** A single file, a folder, or a folder of folders. Type the full path.
- **The work title.** Optional; taken from the filename if you leave it blank.
- **Count collectives as actors.** Whether a group — a family, a crew, a faction — is a character in its own right, or only the people named in it. Leave it off for a novel. Turn it on only where a group genuinely acts as one, as in a serial about rival houses. It applies to the whole project, and snapshots taken either side of a change to it answer different questions and cannot be compared.
- **A role for each document**, plus an optional line to drop everything before. The next chapter is about these.

Create the project writes the file, records your settings, saves your answers about the documents, and reads the text in. It calls no model and costs nothing. Analysing is a separate act, and deliberately so — you can build a project, look at what it holds, and abandon it without having spent anything.

11 Making a project from the command line

The same thing in one line. From inside the folder holding your manuscript:

```
dramatis ingest . --work "My Novel" --creator "Your Name"
```

Or for a single file:

```
dramatis ingest draft.txt --work "My Novel"
```

Dramatis reports what it did:

```
ingested: rev:568a07982816 (4 documents, 3,784 characters, sha256 568a07982816...)
  4 added
  store       dramatis.sqlite  (new project)
  collection  col:my-novel
  work        work:my-novel
  revision    rev:568a07982816

  added     cast.md
  added     chapter-01.md
  added     chapter-02.md
  added     chapter-03.md
```

Four things about `ingest` are worth knowing:

- **It is the only command that creates a project.** Every other command refuses to run rather than quietly starting an empty project you did not ask for — which is what saves you when you run a command from the wrong folder.
- **Running it twice is harmless.** The revision identifier comes from the content of the files, so ingesting unchanged text again changes nothing.
- **It tells you what moved.** On a second ingest, each file is reported as *added*, *changed* or *unchanged*, which is what later lets a comparison say that the graph moved because chapter three was rewritten.
- **Always pass `--work`.** If you ingest `.` without one, the title is derived from a folder name that a bare dot does not supply, and you get a work called *untitled*.

> WHERE THE PROJECT FILE IS LOOKED FOR
>
> Every command except `ingest` searches for `dramatis.sqlite` in the current folder and every folder above it, the way version control finds its own directory. So commands work anywhere inside a project, and `dramatis status` will always tell you which file it found and how.

One project, one collection

A second work ingested into the same project joins the same collection and shares its cast — which is exactly what a series or a shared universe wants:

```
dramatis ingest book-one.txt --work "Meteor Girl" --collection "Golden Age"
dramatis ingest book-two.txt --work "The Spark"        # joins the same collection
```

An unrelated book belongs in its own project file. Dramatis refuses to merge two casts into one namespace, because that is not something you could undo.

12 Saying what each file is

Before it reads anything, Dramatis wants to know what it is looking at. It proposes; you confirm. It never infers a filing convention, because filing conventions are personal and a wrong guess would silently corrupt the reading.

Looking at a folder costs nothing

```
dramatis structure .
```

This reads no model and touches no network. It prints one block per file, each proposal followed by the evidence for it:

```
/writing/lamplighter - 4 documents

  cast.md  (1,022 characters)
    role         unknown
                 needs the text read, not the folder named
    addressing   section
                 blank-line sections
    revision of  (none)
                 no earlier document is named cast.md

  chapter-03.md  (910 characters)
    role         unknown
    ...
```

The three questions

- **Role — narrative or reference?** A *narrative* document enacts the story: chapters, scenes, transcripts. A *reference* document describes it: a cast list, a series bible, an outline, a wiki export. The distinction is not cosmetic — relationships found in the two are kept apart all the way through, and the disagreement between them is one of the most useful things Dramatis will show you (chapter 20).
- **Addressing — how is a position in this file named?** At present Dramatis divides text on blank lines and calls each block a *section*. Richer structures come later.
- **Revision of — is this a later state of another file?** Proposed from the filename and how much text two files have in common, and shown with a percentage so you can judge it.

Answering

In the browser, you answer by clicking the roles in the New project panel. From the command line, you correct and confirm in one go:

```
dramatis structure . --set cast.md=reference --confirm
```

`--confirm` saves your answers against that folder, so you are never asked again; later ingests apply them automatically. It refuses while any document still has an unknown role, because half a map is worse than none. If your mind changes:

```
dramatis structure . --forget
```

Letting a model read the folder for you

For a large or unfamiliar corpus, a model can read the documents and propose roles itself. This is the only part of `structure` that costs anything, and you have to ask for it:

```
dramatis structure . --ask
```

It still writes nothing. You look at the proposals, correct what is wrong, and confirm separately.

Leaving out a preface

Editions of classic texts often bind a critical introduction into the same file as the novel. Its characters are a modern scholar and their colleagues, and they have no business in your graph — and analysing them costs money.

In the New project panel, each document has a field reading *"drop everything before this line"*. Paste the first line of the actual narrative into it. Everything above is dropped before the text is stored, so it never reaches the model at all.

13 Running an analysis

The one step that calls a model, costs money or time, and produces a snapshot. Everything else in Dramatis is free.

You need the identifier of the text revision to read. `dramatis status` lists them:

```
dramatis status
```

```
project     /writing/lamplighter/dramatis.sqlite
            found by searching upward from /writing/lamplighter

settings    collectives_are_actors = false

collection  The Lamplighter's Daughter  (col:the-lamplighter-s-daughter)
registry    4 character(s)

work        The Lamplighter's Daughter  (work:the-lamplighter-s-daughter)
            2 revision(s), 1 snapshot(s)
  snap:bb9660a6fe4b  2026-08-19T07:47:12+00:00  4 characters, 4 relations  draft one
```

Then analyse it:

```
dramatis analyse rev:568a07982816 --label "draft one"
```

Or, with a model on your own machine:

```
dramatis analyse rev:568a07982816 --provider ollama --label "draft one"
```

What comes back:

```
snapshot    snap:bb9660a6fe4b
  revision  rev:568a07982816
  run       run:d5f0082cf9e2
  model     llama3.2:3b
  characters 4
  relations  4
    observed 3  (interaction_passages)
    asserted 1  (asserted_statements)
  rejected   6 of 37 quotations as not verbatim
```

Read that last block carefully; it is the honesty of the tool on display.

- **Observed and asserted are counted separately**, with the basis of each. Three relationships were shown happening in the chapters; one was stated in the cast list. A single total would hide the distinction, and the distinction is the finding.
- **Six quotations were rejected.** The model produced them; they were not found in the text; they were removed along with the claims resting on them. If more than a quarter of them fail, the whole run is refused rather than delivering a plausible but thin graph.

The options that matter

`--provider ollama`	Use a model on this machine. No key, no network, nothing leaves.
`--model`	Name a specific model rather than the provider's default.
`--effort`	How hard the model should work per passage: `low`, `medium` (the default), `high`, `xhigh`, `max`. Higher costs more and reads better. Ollama has no such dial, and Dramatis says so rather than pretending.
`--like SNAPSHOT`	Copy the settings recorded by an earlier snapshot, so a re-run after a rewrite holds the reading still. **The most important option here.**
`--label`	A human name for the snapshot, such as "draft two, after the cut".
`--checkpoint FILE`	Record every model call to a file, so an interrupted analysis of a long novel resumes instead of paying for the same calls twice.

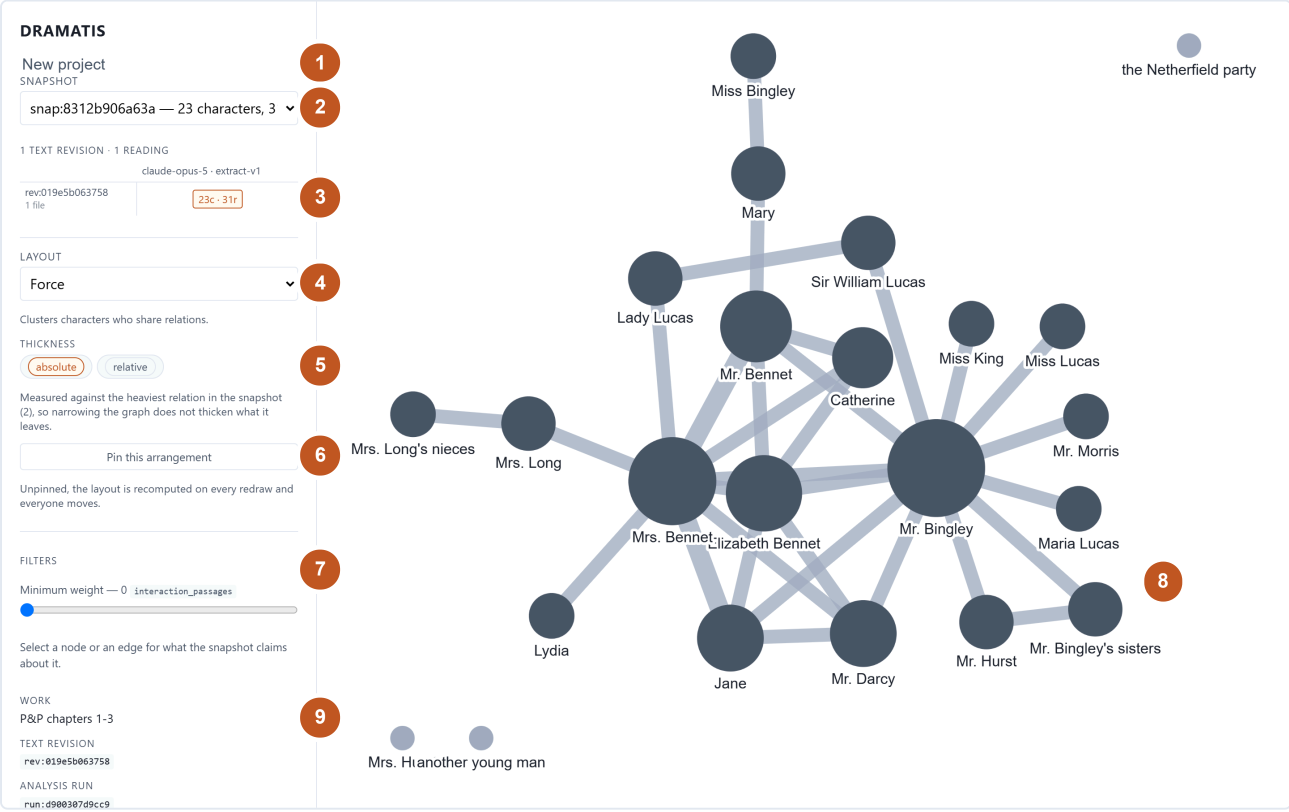
> A CHECKPOINT FILE HOLDS YOUR TEXT
>
> It contains the full passages sent to the model. Keep it beside the project, and never in version control or a shared folder.

14 The window, part by part

From inside your project folder:

```
dramatis serve
```

Then open `http://127.0.0.1:7373`. The server reads only your machine's own loopback address, so nothing on your network can reach it. Stop it with `Ctrl-C`.



The whole interface. One sidebar of controls and one stage. There are no menus, no toolbars and no hidden panels — everything the application can do is on this screen. Shown: the first three chapters of *Pride and Prejudice*, 23 characters and 31 relationships.

1. **New project** — build a project from files, without the terminal.
2. **Snapshot chooser** — every reading the project holds. Switching is instant and free.
3. **The lineage grid** — your snapshots on the two time axes. Click to open; shift-click a second to compare.
4. **Layout** — how the characters are arranged. Five choices, each with a note saying what it is good for.
5. **Thickness** — whether edge width is measured against the whole snapshot or only what is on screen.
6. **Pin** — keep this arrangement so the graph reopens looking the same.
7. **Filters** — narrow by weight, relationship type, or provenance. Only the controls this snapshot can actually use appear.
8. **The graph** — click a circle for a character, a line for a relationship, the background to deselect. Drag to move, scroll to zoom.
9. **Provenance of the picture** — which work, which text revision, which analysis run, which model, which version of the instructions. This is what makes the graph citable.

15 Reading the graph

What size and thickness mean

A circle is a character, sized by how much of the work they carry. A line is a relationship, thickened by its weight. Both are on a square-root scale, and that is a deliberate distortion: interaction counts in fiction are wildly skewed, and on a straight scale the two leads would be drawn as ropes and everyone else as indistinguishable hairs.

A pale circle is a character with no relationships left in view — either they never had any, or your filters have removed them all.

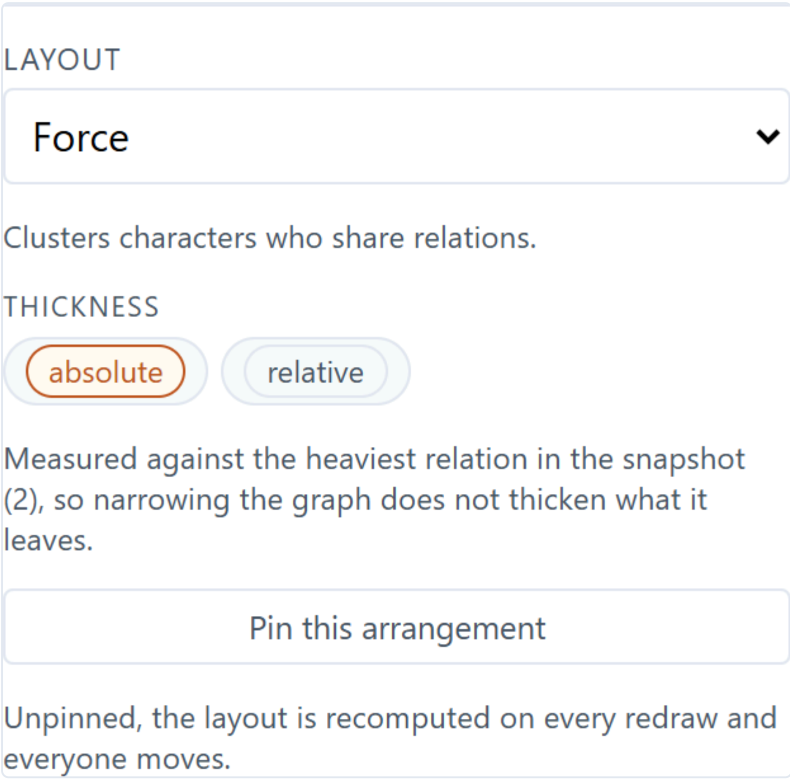
Absolute and relative thickness

Absolute measures every edge against the heaviest relationship in the whole snapshot, so narrowing the graph does not fatten what remains. **Relative** measures against the heaviest edge currently on screen, so a filtered view uses its full range. Use absolute when comparing; relative when studying a corner.

Where a snapshot holds relationships measured on different bases — some counted from passages, some from statements in your notes — Dramatis measures within each basis separately and says so beneath the control. It will not put two incomparable numbers on one scale.

Layouts

Five arrangements: **Force** (the default, clusters characters who share relationships), **Circle**, **Concentric**, **Hierarchy** and **Grid**. Each has a one-line note under the chooser saying what it shows well.



Layout, thickness, and the pin. The note under each control tells you what the setting actually does to the picture, so you do not have to experiment to find out.

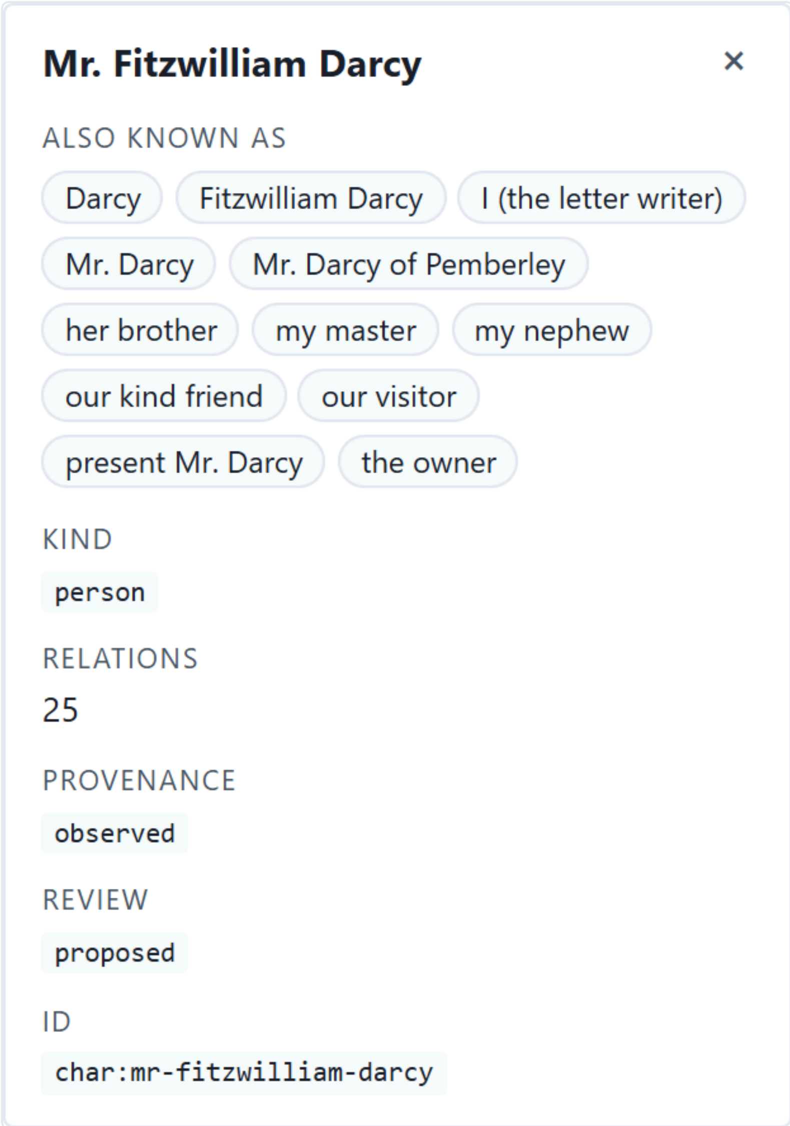
Pinning

A force layout has no memory: every redraw scatters the arrangement you have just finished learning. *Pin this arrangement* keeps it. The graph reopens exactly as you left it, and if you drag a character somewhere better, the new position is kept too.

A pin belongs to one snapshot and lives in your browser, not in the project file. Two consequences: a colleague opening your project sees their own arrangement, and clearing your browser's storage loses the pin but nothing else.

16 Asking a character a question

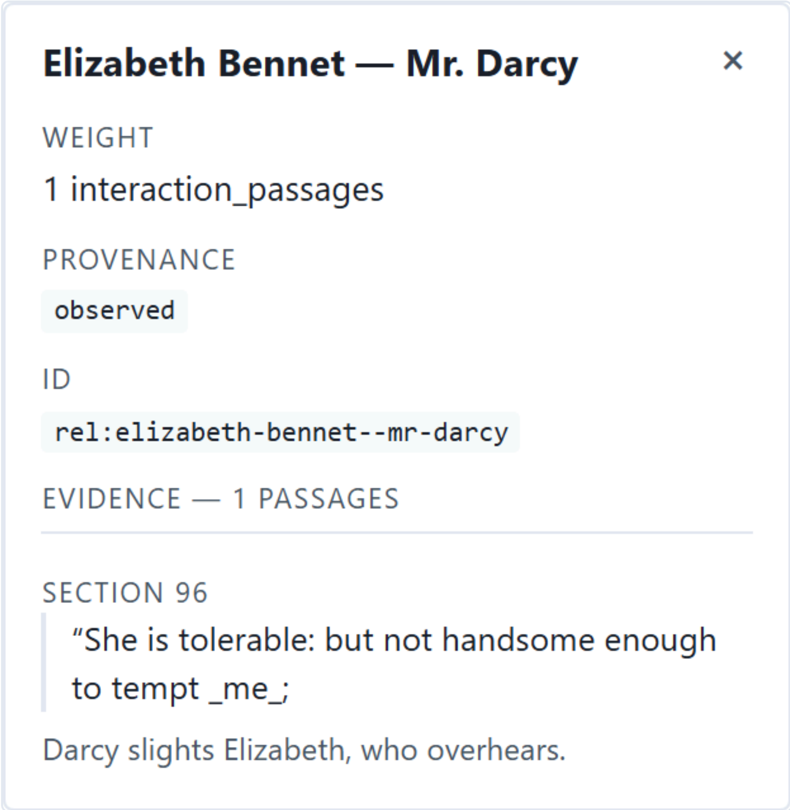
Click a circle. The sidebar fills with what the snapshot claims about that person.



Mr Darcy, from a full-novel reading. The *also known as* list is the work of alias resolution: every surface form the novel used, gathered onto one person. “my master”, “our visitor” and “I (the letter writer)” are all Darcy, and catching that is most of what separates a useful graph from a list of near-duplicate names.

- **Also known as** — every name the text used for them.
- **Kind** — a person, or a collective.
- **Relations** — how many other characters they are connected to.
- **Provenance** — observed, asserted, or human.
- **Review** — how far the claim has travelled through human checking. Everything is `proposed` at present; accepting and correcting arrive in a later version.
- **ID** — the stable identifier, the same across every snapshot in the project.

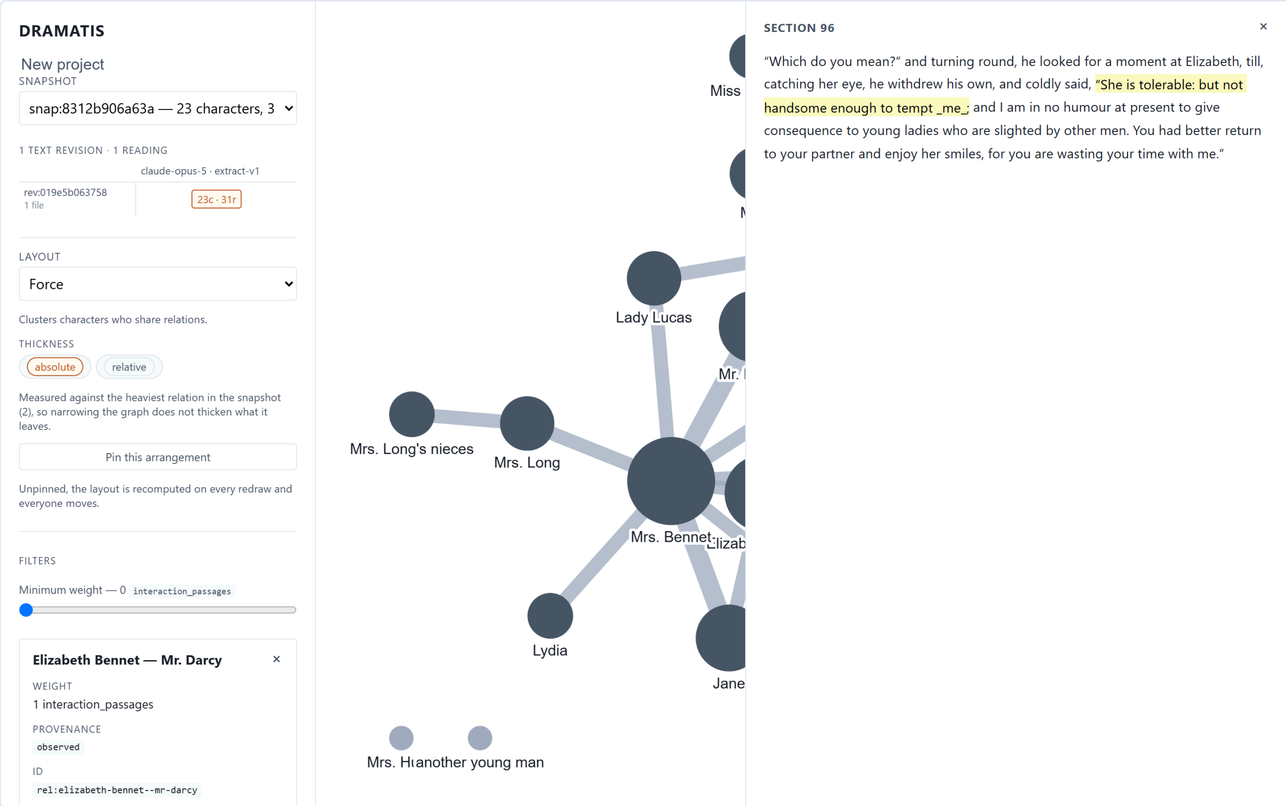
Click a line instead and you get the relationship, with its evidence.



Elizabeth Bennet and Mr Darcy, three chapters in. One passage of contact so far, and the panel names it, quotes it, and says why it counts. Every claim in Dramatis looks like this underneath.

17 Following the evidence into the text

Click any quotation in the evidence list and your source opens beside the graph at that passage, with the quoted words highlighted and scrolled into view.



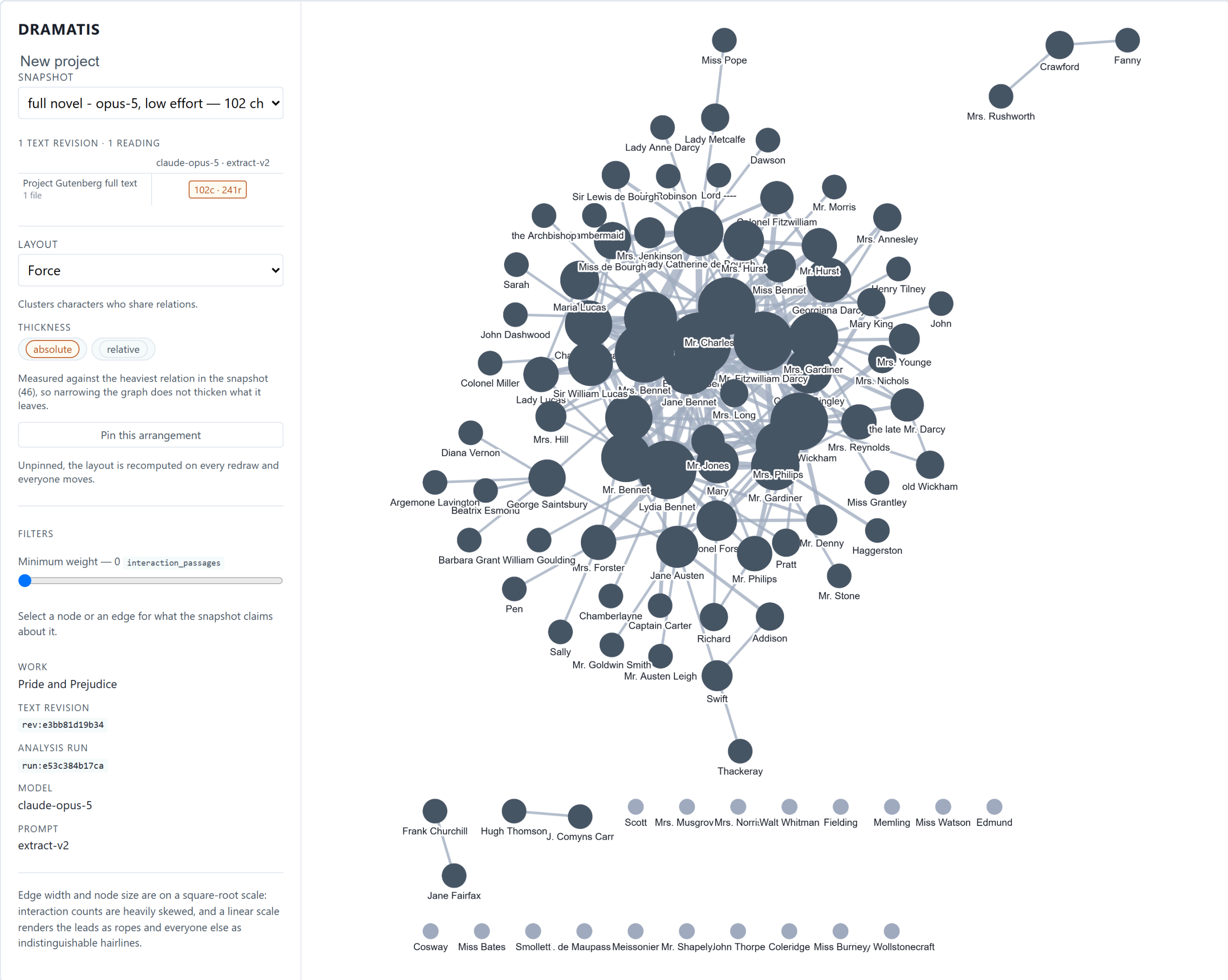
From an edge to a sentence in two clicks. The highlight is computed by the server against the stored text, not searched for by the browser — which is why it works on quotations that span the line breaks of a hard-wrapped file.

The reader is candid about how well it found the passage, and this matters when you have edited the text since the analysis was run:

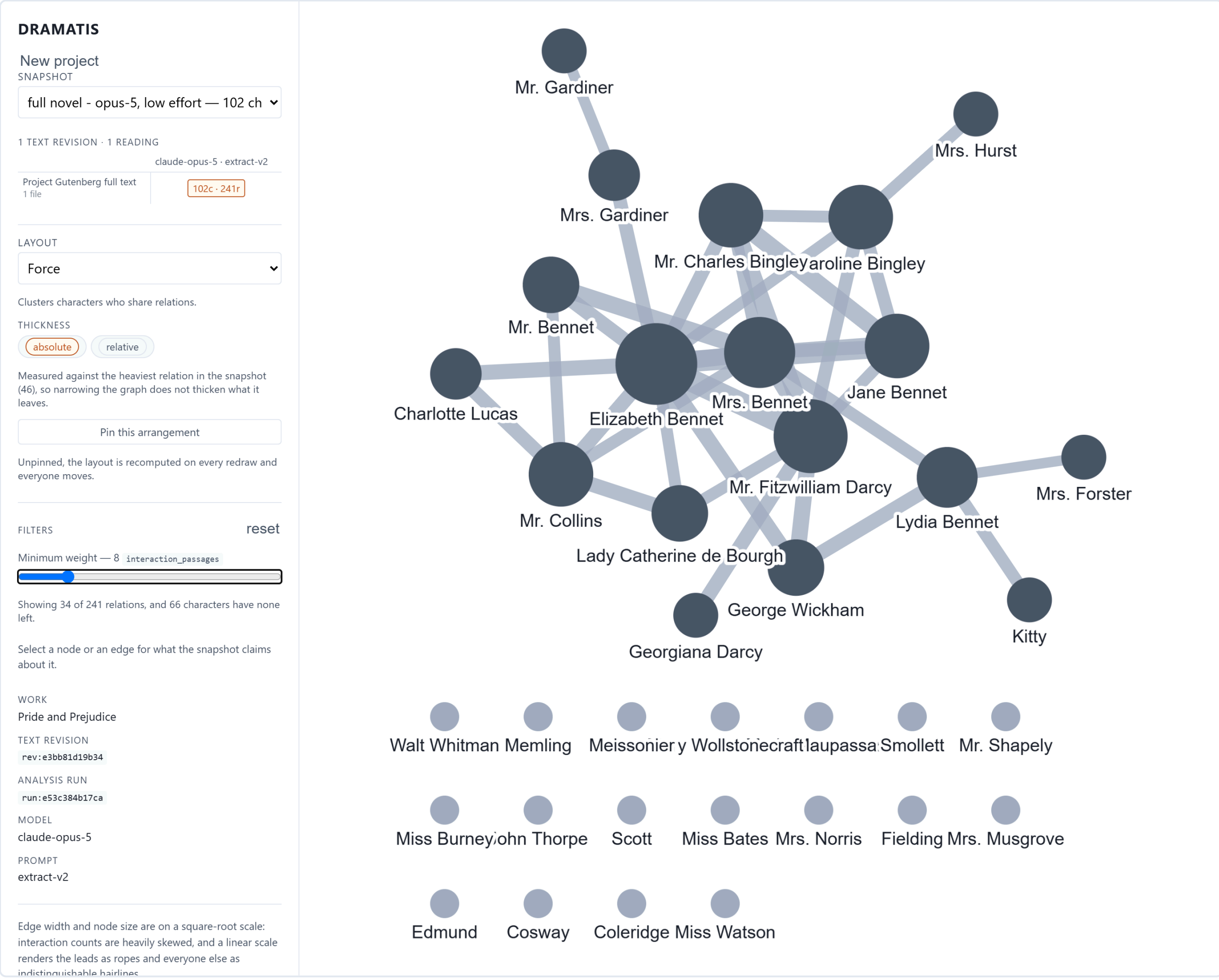
- **Found exactly.** No remark. The quotation is in the current text, word for word.
- **Found approximately.** The highlight is drawn in a different style and a note says the match was fuzzy — you edited inside the quoted sentence, and this is the closest passage. Weigh it accordingly.
- **Moved.** A note says where the evidence used to point. Inserting three paragraphs earlier in a chapter shifts everything after it, and Dramatis simply finds the sentence again.
- **Not found at all.** The passage is shown without a highlight and a plain error says the quotation is no longer in this revision. You rewrote it past recognition. Saying so is the useful answer; silently pointing at whatever scored highest would make every citation in the project unfalsifiable.

18 Narrowing a crowded graph

A whole novel is a hairball. That is not a defect of the drawing — it is what a novel's cast looks like — but it is not readable, and the filters exist to cut through it.



All of *Pride and Prejudice*: 102 characters, 241 relationships. Every walk-on part, every servant, every author named in the Project Gutenberg apparatus at the end of the file. True, and unreadable.



The same snapshot with the weight slider at 8. Thirty-four relationships of 241 remain and the novel appears: the Bennets, the Bingleys, Darcy, Wickham, Collins, Lady Catherine. The tally beneath the slider states exactly what is being hidden — 66 characters now have no relationships left in view.

Three filters, and each appears only if this snapshot has something for it to do:

- **Minimum weight.** A slider. The label names the basis being counted, so you always know what the number means. The graph is rebuilt when you let go, not while you drag.
- **Relationship type.** Chips for whatever vocabulary this analysis produced — kinship, courtship, rivalry, employment. The vocabulary is open, so it varies by work, and no snapshot is forced into a closed list.
- **Provenance.** Observed, asserted, human. Show only what your notes declare, or only what the chapters enact.

Filters are per snapshot and reset when you switch, because “kinship” carried across to a snapshot that never heard the word would silently empty the graph. There is a *reset* control whenever anything is narrowed.

19 Comparing two drafts

The reason the application exists. One snapshot is a picture of a cast; two snapshots are a story about how the cast moved.

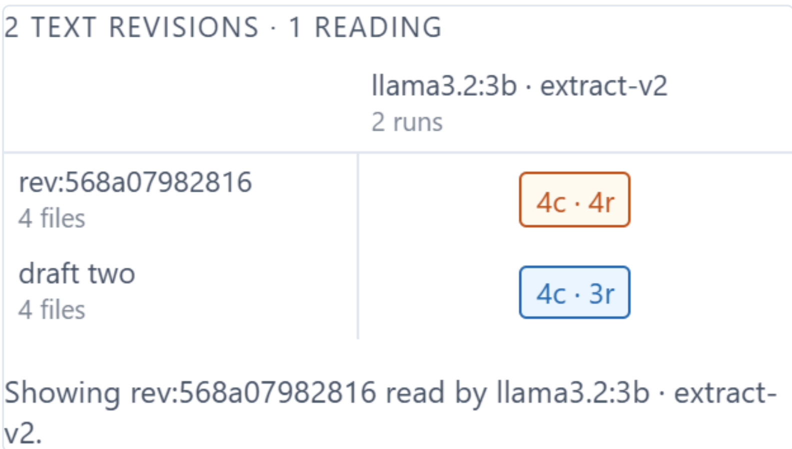
Producing a comparable pair

Three steps, in order:

1. Edit your manuscript in whatever you write in.
2. `dramatis ingest . --work "My Novel"` — records a new text revision and reports which files changed.
3. `dramatis analyse rev:NEW --like snap:OLD` — reads the new text with *exactly* the settings the old snapshot recorded.

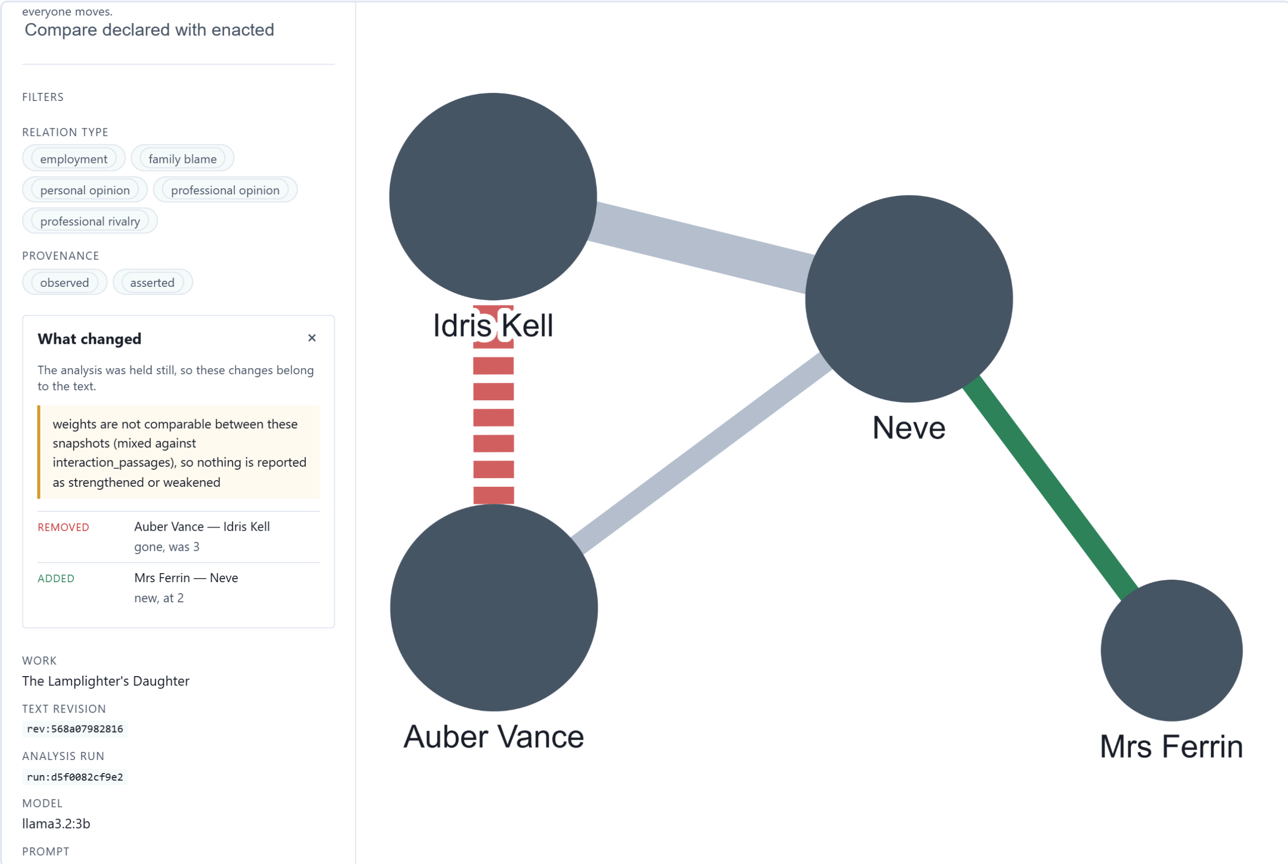
That third step is the one people skip and regret. Passing the same options by hand is not the same claim as copying what the earlier run recorded, and without it the comparison cannot separate your rewrite from a change in the reading. Dramatis prints what it copied, and if you override something explicitly it warns you that the two snapshots will no longer be comparable.

The lineage grid

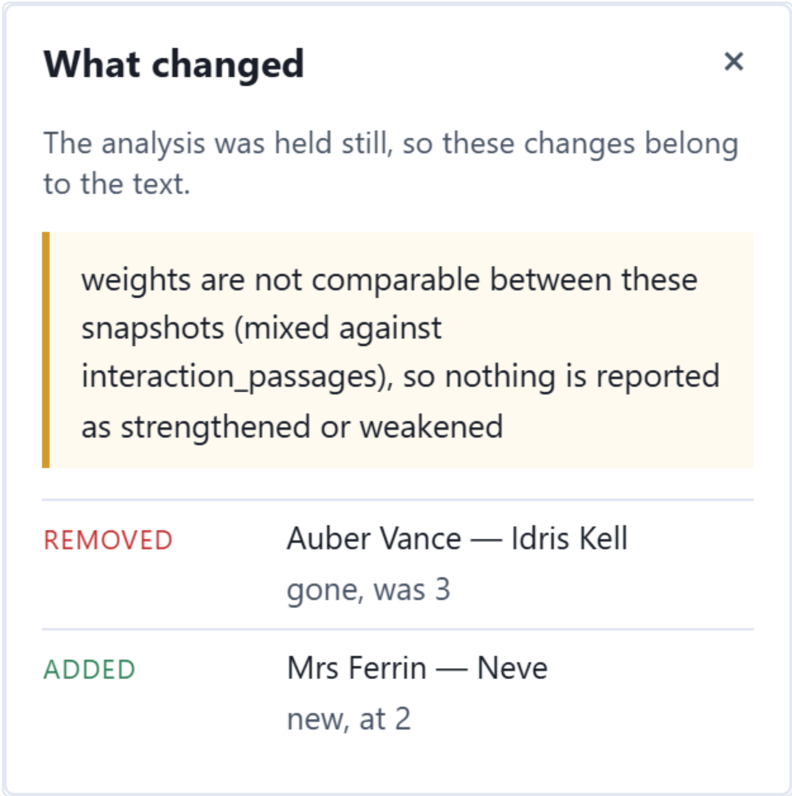


Two drafts read the same way. A row per text revision, a column per reading. Click a cell to open that snapshot; hold Shift and click a second to compare the two. A dot marks a pairing nobody has analysed, which is a different fact from an analysis that found nothing.

The comparison



The comparison as a picture. Everything either snapshot had is drawn, so what was removed is still on screen: red and dashed for gone, green for new, plain grey for unchanged. A graph cannot say “from 25 to 4”, so the same changes are also listed in words beside it.



The same comparison in words. Three things worth noticing, in the order they appear: the attribution sentence at the top; the withheld weight comparison, refused with its reason rather than computed wrongly; and only then the changes themselves.

The colours mean:

- **Added** — new in the later snapshot.
- **Removed** — gone from the later one, and still drawn, faint and dashed, because what disappeared is the half you are least able to reconstruct from memory.
- **Strengthened** — more weight behind it.
- **Weakened** — less.
- **Retyped** — the kind of relationship changed.

Attribution: the first line of the panel

Before it lists anything, the comparison says what the difference can be blamed on. There are three possible answers and you should read the sentence every time:

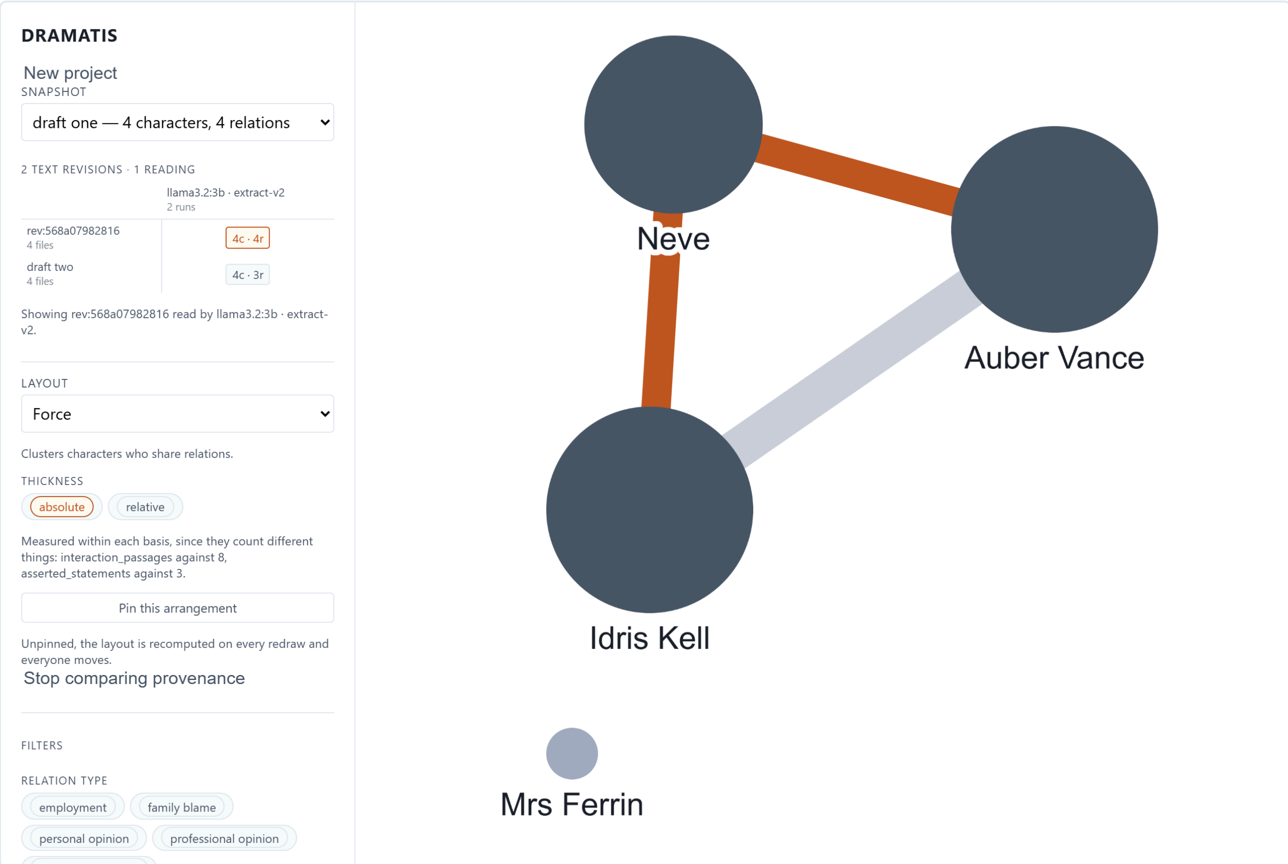
- **The work changed.** Same reading, different text. Everything below is your writing.
- **The reading changed.** Same text, different analysis. Everything below is the model, not you.
- **Both changed.** Nothing below can be credited to either, and the panel says so in a warning colour rather than quietly picking whichever moved more.

Two further refusals keep the list honest. Weights are compared only where both snapshots measured on the same basis; where they disagree, the weight comparisons are withheld and the reason given. And where two characters have been merged into one, the relationships are compared through the merge, so a single act of tidying does not appear as dozens of spurious additions and removals.

20 What you declared against what you wrote

If your project contains reference material — a cast list, a series bible, an outline — Dramatis has read it separately and can put the two readings on top of each other.

The button in the sidebar reads **Compare declared with enacted**. It appears only when the snapshot has both kinds of relationship to compare, and it is never drawn at the same time as a draft comparison: two colour systems answering different questions in one picture would answer neither.



The plan against the page. Orange: two relationships the chapters carry that the cast list never mentions. Faint grey: the one relationship both agree on, receding because it needs no attention. A relationship declared and never enacted would be drawn purple and dashed; this small reading found none, which is itself a finding about the reading rather than about the book.

The two disagreements are the finding. A relationship the bible describes at length and the narrative never dramatises is a scene you meant to write; a pair who carry more of the book than anyone while the bible does not mention them is a subplot that took over while you were not looking. Both are ordinary and both are hard to see from inside the manuscript.

No weights appear anywhere in this panel, and that is deliberate. A relationship your notes state twice is not twice as strong — you repeated yourself. Passages of contact and statements of fact are different quantities, and a column of numbers side by side would invite exactly the comparison the rest of the tool refuses to make.

Relationships that are neither declared nor enacted — ones entered by hand — are counted and named at the top of the panel rather than quietly excluded, so the totals here agree with the totals everywhere else.

21 A cast that spans several works

When a project holds more than one work, the shared character registry becomes useful in its own right:

```
dramatis characters
```

```
Jane Austen - 102 characters across 1 works

  Caroline Bingley  [person]
    also        Caroline, Miss Bingley
    appears in  Pride and Prejudice (12 relations)

  Charlotte Lucas  [person]
    also        Charlotte, Miss Lucas, Mrs. Collins
    appears in  Pride and Prejudice (13 relations)
```

Whoever spans the most works is listed first, because that is the question a shared-universe registry is opened to ask. To see only those:

```
dramatis characters --spanning
```

Two details worth knowing. Appearances are worked out from snapshots rather than stored separately, so they cannot fall out of step — and only the *newest* snapshot of each work is consulted, because a character you cut in the second draft does not appear in that novel any more. A character present in the registry but in no current reading is listed as such rather than left blank, since that is a real state and a blank line reads as a bug. This command calls no model and reaches no network.

22 Every command, at a glance

COMMAND	WHAT IT DOES	MODEL?
`dramatis status`	Which project file is in use, how it was found, and everything in it: settings, collection, works, revisions, snapshots.	No
`dramatis structure`	What a folder appears to hold, with the evidence for each guess. `--set` corrects, `--confirm` saves, `--forget` discards, `--ask` has a model read the documents.	Only with `--ask`
`dramatis ingest`	Reads a file or folder into the project as a text revision. The only command that creates a project.	No
`dramatis analyse`	Reads a stored revision with a model, verifies every quotation, records a snapshot. *(`analyze` also works.)*	**Yes**
`dramatis serve`	Opens the project in your browser on this machine only. Default address `127.0.0.1:7373`.	No
`dramatis characters`	The collection's cast and which works each character appears in.	No
`dramatis validate`	Checks a snapshot file against the published schema and its internal references. Works on documents produced by other tools.	No

Options shared by most commands:

`--store PATH`	Name the project file directly instead of letting Dramatis search for it. Also accepts a PostgreSQL address for shared installations.
`--json`	Machine-readable output on standard output. Every remark, note and warning goes to standard error instead, so the JSON is never contaminated.
`--help`	Full options for any command: `dramatis analyse --help`. The help text is written in the same voice as this manual.

PART IV

A project over time

One short novel, followed from an empty folder to a finished study, in eight milestones. Each gives the reason for the step, the exact commands, and what you should look at afterwards. Follow it with your own work substituted and you will have used every feature Dramatis currently has.

23 Eight milestones with one short novel

The Lamplighter's Daughter. Auber Vance has lit the same round for forty years. Neve, his daughter, keeps the accounts at a chandlery. Idris Kell is the borough inspector who cuts the round in half. There is a cast list. There will be two drafts.

> ABOUT THE SCREENSHOTS IN THIS PART
>
> These readings were produced with a small model running locally (`llama3.2:3b`) so that the whole example could be reproduced free of charge and with nothing leaving the machine. That has a visible consequence, called out at milestone 3, and it is left in rather than tidied away: it is exactly the kind of error you should expect to find and check.

MILESTONE 1 A folder, looked at

Why. Before spending anything, find out what Dramatis thinks your folder holds. This costs nothing and reaches nothing.

```
cd ~/writing/lamplighter
ls
```

```
cast.md   chapter-01.md   chapter-02.md   chapter-03.md
```

```
dramatis structure .
```

WHAT TO LOOK AT

Four documents, every role *unknown*, and under each the reason: "needs the text read, not the folder named". Dramatis will not guess that a file called `cast.md` is a cast list. That refusal is the feature — your filing habits are yours, and a wrong guess here would quietly poison every graph that followed.

MILESTONE 2 The project exists

Why. Answer the role question once, and read the text in. Still no model, still no cost.

```
dramatis serve --store dramatis.sqlite
```

Open `http://127.0.0.1:7373`. The New project panel is already open because there is nothing to show. Type the folder path, press *Read it*, mark `cast.md` as **reference** and the three chapters as **narrative**, give the work a title, and press *Create the project*.

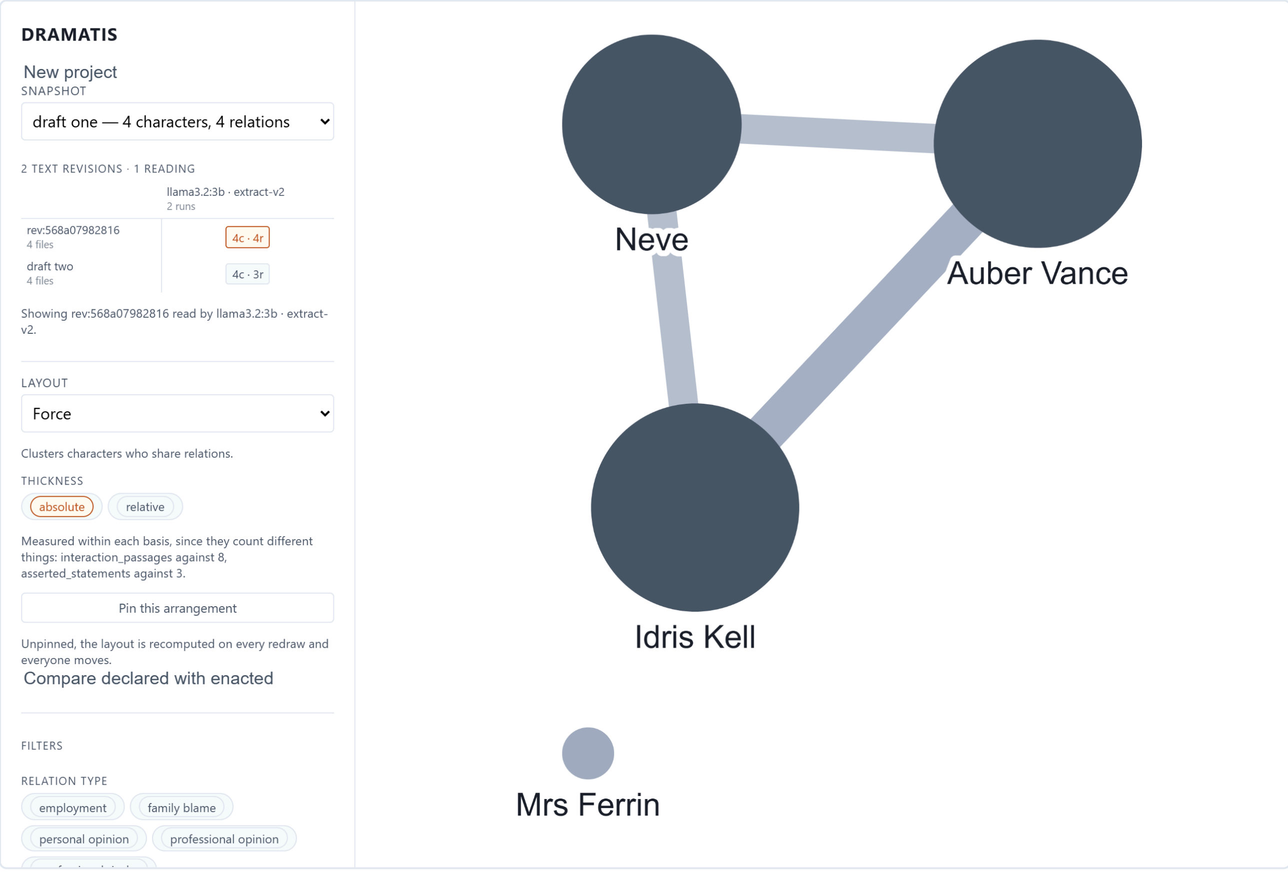
> WHAT TO LOOK AT
> "Ready: 4 documents, 1 as reference material." A file called `dramatis.sqlite` now sits beside your chapters. That single file is the project — the text, the settings, and every reading you will ever run.

MILESTONE 3 The first reading

Why. Now spend something. This is the first and only step that calls a model.

```
dramatis status                       # find the revision identifier
dramatis analyse rev:568a07982816 --provider ollama --label "draft one"
```

```
snapshot    snap:bb9660a6fe4b
  characters 4
  relations   4
    observed 3  (interaction_passages)
    asserted 1  (asserted_statements)
  rejected    6 of 37 quotations as not verbatim
```



Draft one. Three people and the triangle between them. Mrs Ferrin is in the cast list and in chapter two, but this reading found her sharing no passage of contact with anybody, so she sits alone and pale. Small enough to check by hand — which is the point of starting small.

> **WHAT TO LOOK AT**
>
> Two things. First, the split between *observed* and *asserted*: three relationships were shown happening, one was merely stated. Second — and this is the error promised above — open Auber Vance's panel and look at his aliases. The small local model has folded "Neve Vance" into her father. A larger model does not make this mistake, but the lesson holds: click through to the evidence before you trust a node.

MILESTONE 4 Reading the picture properly

Why. Nothing here costs anything, and it is where most of your time will be spent.

Click the heaviest line — Auber and Idris. The panel gives its weight, its basis, and eight passages of evidence. Click a quotation and the chapter opens beside the graph with the sentence highlighted. Try the layouts. Pin the one you like. Drag Neve somewhere more sensible and watch the position stick.

> **WHAT TO LOOK AT**
>
> The distance between "the graph says these two are close" and "here is the sentence that says so" should be two clicks. If a claim looks wrong, this is how you convict it.

MILESTONE 5 The rewrite

Why. This is the whole reason for the two time axes. You have rewritten chapter three: Auber no longer confronts Idris directly — the confrontation is relayed through Neve. Does the graph show it?

```
# after editing chapter-03.md in your own editor
dramatis ingest . --work "The Lamplighter's Daughter" --label "draft two"
```

```
ingested: rev:3fbe3124d583 (4 documents, 3,869 characters)
  1 changed, 3 unchanged
  against     rev:568a07982816

  unchanged cast.md
  unchanged chapter-01.md
  unchanged chapter-02.md
  changed   chapter-03.md
```

Now re-read it, holding the reading exactly as it was:

```
dramatis analyse rev:3fbe3124d583 --provider ollama --like snap:bb9660a6fe4b --label "draft two"
```

```
note: holding the analysis as recorded by snap:bb9660a6fe4b
      (effort=medium, max_rejection_rate=0.25, target_characters=12000)
```

> **WHAT TO LOOK AT**
> "1 changed, 3 unchanged" — Dramatis knows precisely what you touched. And the note about holding the analysis still is what makes the next milestone mean anything.

MILESTONE 6 The comparison

Why. To find out whether the rewrite did what you intended.

Refresh the browser. The lineage grid now has two rows in one column. Click draft one, then hold **Shift** and click draft two.

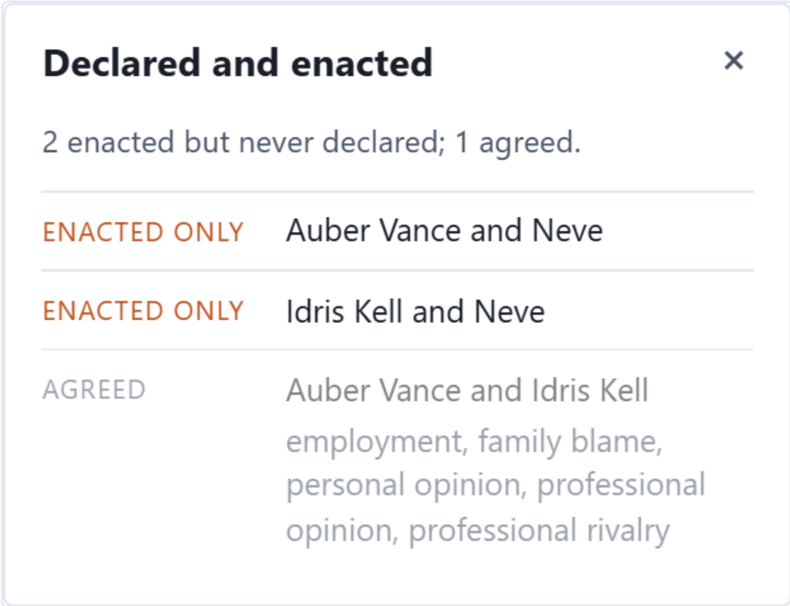
> **WHAT TO LOOK AT**
> The first line of the panel, before anything else. Here it reads *"The analysis was held still, so these changes belong to the text."* That sentence is what licenses everything below it, and you get it only because milestone 5 used `--like`. The second line is the tool declining to do something: the two snapshots weigh their relationships on different bases, so nothing is reported as strengthened or weakened rather than a wrong number being reported confidently. Then read the changes and ask whether they match the edit you made.

MILESTONE 7 The plan against the page

Why. You have kept a cast list. It makes claims. Do the chapters bear them out?

With a snapshot open, press **Compare declared with enacted** in the sidebar.

> **WHAT TO LOOK AT**
> The summary line: here, *"2 enacted but never declared; 1 agreed."* Two of the three relationships the chapters carry are missing from the cast list — an outline that has fallen behind the writing, which is the ordinary condition of an outline. Anything purple and dashed would be the opposite case, a relationship described and never dramatised. Whether either is a gap to fill or a note to delete is a judgement only you can make; Dramatis's job is to put it in front of you.



Disagreements first, agreement last and faint. No weights appear anywhere here: passages of contact and statements in a cast list are different quantities, and printing them side by side would invite a comparison that means nothing.

MILESTONE 8 Keeping it

Why. The project file is now a piece of research with citable results. Treat it like one.

```
cp dramatis.sqlite ~/backups/lamplighter-2026-08-19.sqlite
```

That is the entire backup procedure. To hand the study to a colleague, send them that file: they can open every snapshot, follow every quotation into the text, and compare every draft, with no key, no network and no account. To cite a result, quote the snapshot identifier together with the text revision and the analysis run beneath the sidebar — the three together make the reading reproducible.

What a longer project looks like

Beyond eight milestones the pattern simply repeats. A realistic sequence for a novel written over a year:

WHEN	WHAT YOU DO	WHAT IT COSTS
At the end of each draft	Ingest, then analyse with `--like` the previous snapshot.	One analysis
After a structural cut	Ingest and analyse; compare against the snapshot before the cut.	One analysis
Whenever the cast list changes	Re-ingest and re-analyse; check declared against enacted again.	One analysis
Once, out of curiosity	Re-analyse an *old* revision with a better model, to see how much of a past finding was the reading rather than the writing.	One analysis
Continuously	Open, read, filter, follow evidence, compare.	Nothing

PART V

Living with it

Where the files are, how to keep them safe, what it costs, what it promises about privacy, what to do when something breaks, what is genuinely not built yet, and how to remove every trace of it.

24 Where everything lives

WHAT	WHERE	MATTERS?
Your project	`dramatis.sqlite`, in your manuscript folder unless you named it otherwise	**Everything.** Back this up.
Your manuscript	Wherever you keep it. Dramatis never writes to it.	Yes, but it is yours already
The application	The folder you cloned into, and its `.venv`	No — re-installable in five minutes
Checkpoint files	Wherever you pointed `--checkpoint`	Contain your text; delete when the run is done
Pinned layouts	Your browser's local storage	No — cosmetic, and per browser
Your API key	Your shell environment or profile	Treat as a password

Inside the project file

One SQLite database holding your texts (the full content, not a reference to a path on somebody's laptop), the character registry, and every snapshot as a complete, valid, self-contained document. Nothing points outside the file. That is what makes it portable, archivable and citable — and it is why an evidence quotation can still be resolved against the exact text it was drawn from ten years from now.

When one file is the wrong shape

For several people reading one corpus, or a container with no persistent disk, point `--store` at a PostgreSQL address instead. Nothing else changes:

```
pip install -e ".[postgres]"
dramatis ingest draft.txt --store postgresql://user:pass@localhost/dramatis --work "My Novel"
```

A store is chosen once, not migrated between backends. The single file remains the default and the shape the application is designed around.

25 Backing up, sharing, archiving, citing

Backing up

Copy the file. Do it whenever you have run an analysis worth keeping — that is the only expensive thing in the project. Dated copies are worth the disk space:

```
cp dramatis.sqlite ~/backups/my-novel-2026-08-19.sqlite
```

Whether the project file belongs in version control alongside your manuscript is your call. It is a binary and it grows, but it is also the record of your study.

Sharing

Send the file. Your recipient needs Dramatis installed but needs no key, no account and no network to read everything in it: the graphs, the evidence, the source text, the comparisons. This follows from the rule that reading never requires a model.

> THE FILE CONTAINS YOUR MANUSCRIPT
> Document contents are stored inside the project so that evidence can always be resolved. Sending the project file sends the text. That is usually what you want and occasionally very much not.

Archiving and citing

A snapshot is a complete document conforming to a published, separately versioned schema, so it can be deposited and read by other tools without Dramatis. To make a result citable, name three things together:

- the **snapshot** identifier (for example `snap:bb9660a6fe4b`),
- the **text revision** it read,
- the **analysis run** that read it — which carries the model and the version of the instructions.

All three are in the sidebar under any open snapshot. Given them and the project file, another scholar can reproduce exactly what you saw.

26 What it costs and how long it takes

Only `analyse` costs anything, and `structure --ask` if you use it. Everything else — ingesting, browsing, filtering, comparing, exporting, validating — is free and offline, permanently.

	HOSTED MODEL	LOCAL MODEL
Three chapters	Pennies; under a minute	Free; a few minutes
A full novel	A few dollars at medium effort; several minutes	Free; hours on an ordinary laptop
Every re-read	The same again — the work is not cached between runs	The same again

Three ways to spend less:

- **Exclude what you do not need.** A critical preface bound into the same file has its own cast, and dropping it before analysis costs nothing and saves a real fraction of a long book. See chapter 12.
- **Use `--checkpoint` for long runs.** An interrupted novel resumes instead of paying for the same calls again.
- **Start at low effort.** Read a cheap pass, decide whether the shape is right, then pay for a careful one.

27 Privacy, in detail

Precisely what leaves your machine, and when.

ACTION	WHAT LEAVES YOUR MACHINE
`ingest`, `status`, `validate`, `characters`, `serve`	**Nothing.** Ever. These work offline and always will.
`structure` without `--ask`	Nothing.
`structure --ask`	The opening of each document, to your chosen provider.
`analyse` with a hosted model	Your text, in passages, to that provider and nowhere else.
`analyse --provider ollama`	Nothing, provided Ollama is on this machine — and Dramatis warns you if it is not.

Four further guarantees:

- **No telemetry, no analytics, no update checks, no crash reports.** There is no code in Dramatis that contacts Kestora Labs.
- **The browser interface binds to this machine only.** Serving it to the network is possible, deliberately awkward, and warned about in plain words.
- **The server refuses writes from other websites.** Any page you have open could send a command to an address on your own machine; every write is checked and a request from elsewhere is refused before it reaches anything.
- **One place does networking.** The whole application makes outbound connections from a single small module per provider — the property is checkable by anyone who wants to read the source, which is why it is stated this way.

28 When something goes wrong

WHAT YOU SEE	WHAT IT MEANS, AND WHAT TO DO
`dramatis: command not found`	The virtual environment is not active. Run the `activate` line from chapter 6 step 2 in this terminal window.
`error: no project file found`	You are outside the project. `cd` into the manuscript folder, or pass `--store` with the path. Dramatis refuses rather than creating an empty project — deliberately.
A page saying the client has not been built	The two `npm` commands from chapter 6 step 4 were not run.
Analysis fails on a credential	`ANTHROPIC_API_KEY` is not set in *this* terminal window. It does not survive closing the window unless you put it in your shell profile.
`too many quotations failed verification`	More than a quarter of what the model produced was not in your text, so the whole run was refused rather than delivered thin and misleading. Try a better model or a higher effort. If it persists, the text may have an unusual encoding.
Two names that are obviously one person	Alias resolution missed a merge. A more capable model fixes most cases. Merging by hand is a Phase 5 feature and is not built yet.
A character you do not recognise	Usually front or back matter: a Gutenberg header, a translator's note, an introduction. Exclude it (chapter 12) and analyse again.
"This quotation is not in this revision"	Working as designed. You edited the sentence after the analysis. Re-analyse the new revision when you are ready.
The comparison says it cannot attribute the change	Both the text and the reading moved. Re-run the newer analysis with `--like` the older snapshot and compare again.
An analysis was interrupted	Run it again with the same `--checkpoint` file and it resumes from where it stopped.
`note: skipped ... : not a text file`	Informational. Dramatis reads `.txt`, `.md`, `.markdown` and `.text`; anything else in the folder is named and left out rather than dropped silently.

29 What Dramatis cannot do yet

Named honestly, so you can plan around them, and listed in roughly the order they are expected to arrive.

NOT YET	WHAT IT WILL BE
Correcting the graph by hand	Accepting, rejecting and correcting individual claims, with your corrections surviving re-analysis rather than being silently overwritten.
Merging and splitting characters	Joining two nodes that are one person, or separating one that is two, with the decision recorded in the graph. Today this is entirely the model's call.
A continuity report	Characters renamed between drafts, listed with references, so a rename does not read as a death and a birth.
Confidence shown on screen	Low-confidence relationships drawn distinctly from well-evidenced ones.
Exports	GraphML, GEXF and CSV for Gephi and similar tools; evidence as W3C Web Annotation. Today the project file and its JSON are the export.
Multiple editions of one work	Comparing translations or editions inside a single collection.
A double-clickable application	A signed desktop installer for macOS and Windows, with no terminal at all.
Editing the instructions	Per-project prompts, so a scholar can state and publish exactly how their corpus was read, with an evaluation harness to show the change was an improvement.

30 Uninstalling, completely

Nothing is hidden anywhere on your system. Removing Dramatis is removing two folders, and you decide separately what to do with your projects.

1 · Decide about your projects first

Your `dramatis.sqlite` files are in your own manuscript folders and are not touched by any of the steps below. Each holds analyses you paid for. Copy them somewhere safe if there is any chance you will want them, or delete them deliberately.

```
# find every project on the machine (macOS and Linux)
find ~ -name "dramatis.sqlite" 2>/dev/null
```

```
# Windows PowerShell
Get-ChildItem -Path $HOME -Filter dramatis.sqlite -Recurse -ErrorAction SilentlyContinue
```

2 · Remove the application

Delete the folder you cloned into. That is the virtual environment and all of it:

```
rm -rf ~/dramatis          # macOS and Linux
```

```
Remove-Item -Recurse -Force $HOME\dramatis      # Windows PowerShell
```

3 · Remove what you added for it

- **Your API key.** Delete the `ANTHROPIC_API_KEY` line from your shell profile, and revoke the key itself in the provider's console — deleting the line does not disable the key.
- **Checkpoint files.** Any file you passed to `--checkpoint`. These contain your text.
- **Ollama**, if you installed it only for this. Uninstall it like any other application; `ollama rm llama3.1` first reclaims the model's disk space.
- **Python and Node**, if you installed them only for this and nothing else on your machine uses them. Most people should leave them.
- **Pinned layouts** live in your browser's storage for `127.0.0.1:7373` and disappear when you clear site data. They are cosmetic.
- **The Docker image**, if you used it: `docker rmi dramatis`.

> THERE IS NOTHING ELSE
>
> No system-wide configuration, no registry entries, no background service, no login item, no account to close, and no remote copy of anything — because nothing was ever sent anywhere except to the model provider you chose, at the moment you asked.

END MATTER

Glossary and credits

A Glossary

Alias	A surface form of a name: "Lizzy" for Elizabeth Bennet. Gathering aliases onto one character is *alias resolution*, and it is the hardest thing the pipeline does.
Analysis run	One reading: a model, a version of the instructions, and the settings used.
Asserted	A claim your reference material states, as opposed to one the narrative enacts.
Attribution	A comparison's verdict on *what* a difference can be blamed on: the work, the reading, or neither.
Checkpoint	A file recording every model call, so an interrupted analysis resumes rather than paying twice. Contains your text.
Collection	The circle inside which characters are shared. One per project.
Document	One file in a project, with a role: narrative or reference.
Evidence	A quotation with a locator and often a note, attached to a relationship. Verified verbatim against the source or discarded.
Ingest	Reading text into a project as a fingerprinted revision. Never calls a model.
Lineage	A work's snapshots arranged on the two time axes.
Locator	Where in a document something is: an ordered path of typed segments. The types are data supplied per work, so the schema names no chapter, panel or scene.
Observed	A claim the narrative enacts on the page.
Provenance	Where a claim came from: observed, asserted, or human.
Re-anchoring	Finding a quotation again after the text around it has been edited — exactly, then by surrounding context, then approximately, and never silently.
Region	A marked part of a document, which may be excluded so it is never stored or analysed. What a preface goes in.
Review status	How far a claim has travelled through human checking: proposed, accepted, corrected, rejected. Everything is *proposed* today.
Role	Whether a document enacts the story (narrative) or describes it (reference).

Schema	The published, separately versioned definition of a Dramatis document, so other tools can read and write the format without running Dramatis.
Segment	A division of a text. Dramatis currently divides on blank lines and calls each block a *section*.
Snapshot	A graph bound to one text revision and one analysis run. Immutable.
Structure map	Dramatis's proposals about a folder, with evidence, awaiting your confirmation.
Text revision	The exact fingerprinted state of a work's files at one moment.
Weight basis	What a weight counts. Weights are comparable only within a shared basis, and Dramatis refuses to compare across bases rather than producing a wrong answer.

B Licences and credits

Dramatis ships no model and no credentials. Whichever model you use is yours, under that provider's terms.

The screenshots of *Pride and Prejudice* are from the Project Gutenberg text, which is in the public domain. *The Lamplighter's Daughter* is synthetic, written for this repository's test fixtures, and deliberately short.

Dramatis — The Manual. Describes version 0.1.0.dev0, built through Phase 4 of the roadmap. Everything stated here was checked against the running application; every screenshot is of a real analysis of a real text.

Further reading in the repository: `docs/schema.md` for the document format, `AI/ROADMAP.md` for the design specification and the invariants every change must respect, and `DECISIONS.md` for the reasoning behind the choices that shaped it.